📊 RELATÓRIO EXECUTIVO COMPLETO - PLATAFORMA PRISMA 888

Versão: 2.0 | **Data:** 29/01/2026 | **Classificação:** Documento Estratégico
Autor: Sistema de Análise Automática | **Projeto:** PRISMA888FORBES

📋 ÍNDICE

1. Visão Executiva
2. Para Quem Serve
3. Arquitetura Técnica
4. Módulos da Plataforma
5. Sistema de Inteligência Artificial
6. Estrutura de Dados
7. Mapa de Rotas e Funcionalidades
8. Benefícios por Perfil de Usuário
9. Diferenciais Competitivos
10. Roadmap e Evolução

1. Visão Executiva

1.1 O que é o PRISMA 888?

O **PRISMA 888** é uma plataforma de **Inteligência Política Enterprise** que combina Big Data, Análise Geoespacial e Inteligência Artificial para transformar a gestão de campanhas eleitorais e mandatos políticos.

> **Missão:** Democratizar o acesso à inteligência de dados políticos, permitindo que campanhas de todos os tamanhos tomem decisões estratégicas baseadas em dados reais e análises automatizadas por IA.

1.2 Proposta de Valor

Problema	Solução PRISMA 888
Decisões baseadas em "achismo"	**Dados reais** do TSE + análise territorial
Campanhas reativas	**Planejamento estratégico** gerado por IA
Falta de controle sobre o mandato	**Radar de Promessas** com verificação fiscal
Equipes desconectadas	**Plataforma centralizada** com tarefas e métricas
Análise manual demorada	**Crews de IA** que analisam em minutos

1.3 Stack Tecnológica

```
┌─────────────────────────────────────────────────────────────┐
│                    PRISMA 888 PLATFORM                      │
├─────────────────────────────────────────────────────────────┤
│  FRONTEND                                                   │
│  ├─ Next.js 15.3.6 (App Router)                             │
│  ├─ React 19 + TypeScript                                   │
│  ├─ TailwindCSS + Shadcn/UI                                 │
│  ├─ Leaflet (Mapas Interativos)                             │
│  └─ Phosphor Icons                                          │
├─────────────────────────────────────────────────────────────┤
│  BACKEND                                                    │
│  ├─ FastAPI (Python 3.12+)                                  │
│  ├─ CrewAI (Orquestração de Agentes)                        │
│  ├─ DeepSeek (LLM Principal)                                │
│  ├─ Tavily (Search API)                                     │
│  └─ OpenTelemetry (Observabilidade)                         │
├─────────────────────────────────────────────────────────────┤
│  DATABASE                                                   │
│  ├─ Supabase (PostgreSQL + RLS)                             │
│  ├─ Vector Embeddings (RAG)                                 │
│  └─ Storage (PDFs, Documentos)                              │
└─────────────────────────────────────────────────────────────┘
```

2. Para Quem Serve

2.1 Público-Alvo Primário

🎯 Candidatos a Prefeito/Vereador

- Planejamento estratégico de campanha
- Análise de votos por seção eleitoral
- Geração automática de estratégias e táticas

🎯 Políticos em Exercício de Mandato

- Monitoramento de promessas de campanha
- Cruzamento com execução orçamentária real
- Verificação de cumprimento na mídia

🎯 Coordenadores de Campanha

- Dashboard centralizado de métricas
- Gestão de equipe e tarefas
- Mapa interativo com inteligência territorial

2.2 Público-Alvo Secundário

📊 Consultorias de Marketing Político

- White-label para múltiplos clientes
- Geração de relatórios automatizados
- Análise comparativa de candidatos

📰 Veículos de Comunicação

- Radar de Promessas para jornalismo investigativo
- Dados verificáveis de execução fiscal
- Fact-checking automatizado

3. Arquitetura Técnica

3.1 Estrutura de Diretórios

```
PRISMA888FORBES/
│
├── 📁 frontend/                  # Next.js 15 Application
│   ├── src/
│   │   ├── app/                  # App Router (Rotas)
│   │   │   ├── admin/            # Painel Administrativo
│   │   │   ├── campaign/[id]/    # Dashboard do Candidato
│   │   │   ├── login/            # Autenticação
│   │   │   └── api/              # API Routes
│   │   ├── components/           # Componentes React
│   │   │   ├── admin/            # Componentes Admin
│   │   │   ├── campaign/         # Componentes Campanha
│   │   │   ├── dashboard/        # Widgets do Dashboard
│   │   │   ├── map/              # Mapa Interativo
│   │   │   ├── tasks/            # Sistema de Tarefas
│   │   │   └── ui/               # Biblioteca UI (28 componentes)
│   │   └── lib/                  # Utilitários e Supabase Client
│   └── public/                   # Assets Estáticos
│
├── 📁 backend/                   # FastAPI + CrewAI
│   └── src/
│       ├── api/                  # Endpoints REST (14 arquivos)
│       │   ├── agents.py         # CRUD de Agentes IA
│       │   ├── campaign.py       # Gestão de Campanhas
│       │   ├── radar_premium.py  # Sistema Radar (42KB, 1175 linhas)
│       │   ├── strategies.py     # Estratégias Geradas
│       │   ├── tasks.py          # Tarefas Táticas
│       │   └── ...
│       ├── crew/                 # Sistema de Agentes IA
│       │   ├── genesis_crew.py   # Crew Principal (58KB, 1293 linhas)
│       │   ├── radar_crew.py     # Crew do Radar
│       │   ├── tools.py          # Ferramentas RAG
│       │   └── data_tools.py     # Ferramentas de Dados
│       ├── services/             # Lógica de Negócio (8 arquivos)
│       └── workers/              # Processamento Assíncrono
│
├── 📁 migrations/                # Scripts SQL (21 arquivos)
│   ├── create_cities_politicians.sql
│   ├── create_promises_tables.sql
│   ├── create_radar_premium.sql
│   └── ...
│
├── 📁 docs/                      # Documentação (5 arquivos)
├── 📁 kestra_flows/              # Workflows de Automação
└── 📁 infrastructure/            # Configs de Deploy
```

3.2 Fluxo de Dados Principal

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  CANDIDATO   │ ──▶ │  FRONTEND    │ ──▶ │  BACKEND     │
│  (Browser)   │    │  (Next.js)   │    │  (FastAPI)   │
└──────────────┘    └──────────────┘    └──────────────┘
                                              │
                                              ▼
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  SUPABASE    │ ◀──▶ │  CREW AI     │ ◀──▶ │  DEEPSEEK    │
│ (PostgreSQL) │    │ (Agentes IA) │    │  (LLM)       │
└──────────────┘    └──────────────┘    └──────────────┘
```

---

4. Módulos da Plataforma

4.1 🏠 Dashboard Principal

Arquivo: `frontend/src/app/campaign/[id]/dashboard/page.tsx`

Funcionalidades:

- **Seções Mapeadas:** Contador dinâmico de locais de votação cadastrados
- **Votos Totais:** Soma real de votos da última eleição (dados TSE)
- **Insights Recentes:** Estratégias geradas pela IA nos últimos 7 dias
- **Tarefas Pendentes:** Contador de ações aguardando execução

Widgets Integrados:

Widget	Arquivo	Função
Resultados Eleitorais	`election-results-widget.tsx`	Histórico de votos por candidato
Prioridades Estratégicas	`strategic-priorities-widget.tsx`	Top estratégias ativas
Diagnósticos Recentes	`recent-diagnoses-widget.tsx`	Últimas análises da IA
Mapa Eleitoral	`ElectoralMapFull.tsx`	Visualização geoespacial

4.2 🗺️ Mapa Interativo (Enterprise)

Arquivo: `frontend/src/components/campaign/ElectoralMapFull.tsx`

Tecnologia: React Leaflet (SSR disabled via `next/dynamic`)

Funcionalidades:

- **Pins Coloridos por Performance:**
  - 🔴 Vermelho: Baixa votação (< 30% do esperado)
  - 🟡 Amarelo: Votação mediana (30-60%)
  - 🟢 Verde: Alta votação (> 60%)
- **Painel Lateral ("Sheet"):**
  - Ranking de votos por local
  - Dados do candidato e concorrentes
  - Cálculo de "Market Share" (%)
- **Ações de Guerrilha:**
  - Botão de ação tática por local
  - Integração com API `/tactical_action`
- **Notas no Mapa:**
  - Post-its virtuais por localização
  - Colaboração em tempo real

Tabelas Relacionadas:

- `locations`: Geometria e metadados dos locais
- `location_results`: Votos reais por candidato
- `map_notes`: Anotações dos usuários

4.3 📡 Radar de Promessas (Premium)

Arquivo Principal: `backend/src/api/radar_premium.py` (1.175 linhas)

O Radar é o módulo mais complexo do sistema, executando análise em **3 fases**:

Fase 1: Extração de Promessas

```
Input:  PDF do Plano de Governo
Process: OCR + IA (RadarCrew)
Output: Lista estruturada de promessas
```

- **Endpoint:** `POST /api/campaigns/{id}/radar/{mandate_id}/phase1`
- **Agente:** `radar-extrator-promessas`
- **Tabela:** `promises`

Fase 2: Cruzamento Fiscal

```
Input:  Promessas + Dados do TCESP
Process: Matching por palavras-chave + IA
Output: Relatório de execução orçamentária
```

- **Endpoint:** `POST /api/campaigns/{id}/radar/{mandate_id}/phase2`
- **Agente:** `radar-fiscal-verbas`
- **Fonte:** API TCESP (Tribunal de Contas SP)
- **Tabela:** `municipal_expenses`, `promise_budget_summaries`

Fase 3: Varredura de Mídia

```
Input:  Lista de promessas
Process: Busca web via Tavily + análise IA
Output: Notícias e verificações sobre cada promessa
```

- **Endpoint:** `POST /api/campaigns/{id}/radar/{mandate_id}/phase3`
- **Agente:** `radar-google-scanner`
- **Ferramenta:** `TavilySearchTool`
- **Tabela:** `radar_executions`

4.4 🤖 Genesis Crew (Sistema de Agentes IA)

Arquivo Principal: `backend/src/crew/genesis_crew.py` (1.293 linhas)

Conceito: O Genesis Crew é uma "Sala de Guerra" virtual onde múltiplos agentes de IA colaboram para gerar planos estratégicos completos.

Arquitetura de Agentes:

```
┌──────────────────────────────────────────────────────────────┐
│                        GENESIS CREW                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐    │
│   │ ANALISTA     │──►│ ESTRATEGISTA │──►│ PLANEJADOR   │    │
│   │ De Dados     │   │  Político    │   │  Tático      │    │
│   └──────────────┘   └──────────────┘   └──────────────┘    │
│        │                  │                  │              │
│        ▼                  ▼                  ▼              │
│   ┌──────────────────────────────────────────────────┐      │
│   │                    OUTPUT FINAL                  │      │
│   │  - Plano Estratégico (Markdown)                  │      │
│   │  - Estratégias (JSON estruturado)                │      │
│   │  - Tarefas Táticas por estratégia                │      │
│   └──────────────────────────────────────────────────┘      │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Ferramentas Disponíveis:

Ferramenta	Arquivo	Função
campaign_vector_search	`tools.py`	Busca semântica RAG em documentos
campaign_stats	`tools.py`	Estatísticas de votos e performance
TavilySearchTool	`radar_crew.py`	Busca web avançada

Modelos de IA:

- **LLM Principal:** DeepSeek (via HTTP direto)
- **Fallback:** Configurável por agente
- **Temperature:** 0.7 (padrão, customizável)

4.5 📋 Plano Estratégico

Arquivo: `frontend/src/app/campaign/[id]/plan/page.tsx`

O que contém:

- **Visão Geral:** Resumo executivo da campanha
- **Estratégias por Pilar:**
  - Mobilização de Base
  - Comunicação Digital

  - Corpo a Corpo
  - Alianças Políticas

- **Tarefas Táticas:** Ações concretas para cada estratégia
- **Status de Execução:** Sugerida → Aprovada → Publicada → Executada

Fluxo de Aprovação:

```
[IA Sugere] ──▶ [Coordenador Revisa] ──▶ [Aprovação] ──▶ [Publicação para Equipe
```

4.6 ✅ Kanban de Tarefas

Arquivo: `frontend/src/app/campaign/[id]/tasks/page.tsx`

Funcionalidades:

- Board estilo Kanban (To Do / Doing / Done)
- Tarefas geradas automaticamente pela IA
- Atribuição a membros da equipe
- Exemplos práticos por tarefa
- Tags por categoria

Estrutura da Tarefa:

```
{
  id: UUID,
  title: string,
  description: string,
  examples: string[],        // Exemplos práticos
  tags: string[],            // Categorização
  pillar: string,            // Pilar estratégico
  phase: string,             // Fase da campanha
  assigned_to: UUID,
  completed: boolean,
  strategy_id: UUID          // Vínculo com estratégia
}
```

4.7 👤 Painel Administrativo

Diretório: `frontend/src/app/admin/`

Módulos Admin:

Rota	Função
`/admin/candidatos`	Gestão de candidatos/campanhas
`/admin/dashboard`	Métricas gerais do sistema
`/admin/agentes`	Editor de Personas IA
`/admin/agentes/biblioteca`	Biblioteca de agentes pré-configurados
`/admin/flows`	Editor de workflows Kestra/N8N
`/admin/radar/politicos`	Base de políticos para monitoramento
`/admin/radar/cidades`	Cadastro de municípios
`/admin/radar/monitoramento`	Painel do Radar Premium

5. Sistema de Inteligência Artificial

5.1 Agentes Principais

🔍 radar-extrator-promessas

- **Função:** Leitura e interpretação de PDFs de Planos de Governo
- **Trigger:** Fase 1 do Radar
- **Output:** JSON estruturado de promessas

💰 radar-fiscal-verbas

- **Função:** Cruzamento de promessas com execução orçamentária
- **Trigger:** Fase 2 do Radar
- **Integração:** API TCESP (Tribunal de Contas SP)

🌐 radar-google-scanner

- **Função:** Varredura de mídia sobre cumprimento de promessas
- **Trigger:** Fase 3 do Radar
- **Ferramenta:** Tavily Search API

📊 analista-de-dados

- **Função:** Análise de resultados eleitorais e tendências

- **Trigger:** Genesis Crew
- **Ferramentas:** campaign_stats, campaign_vector_search

🎯 estrategista-politico

- **Função:** Geração de estratégias baseadas na análise
- **Trigger:** Genesis Crew
- **Output:** Estratégias por pilar

📝 planejador-tatico

- **Função:** Conversão de estratégias em tarefas executáveis
- **Trigger:** Genesis Crew
- **Output:** Tarefas com exemplos práticos

5.2 Editor de Agentes

Arquivo: `frontend/src/components/admin/agent-editor/`

Componentes:

- `AgentForm.tsx` : Formulário principal
- `BasicInfo.tsx` : Nome, role, goal
- `PromptEditor.tsx` : Backstory e instruções
- `ToolsSelector.tsx` : Seleção de ferramentas
- `ComplianceConfig.tsx` : Regras de compliance

Campos Configuráveis:

```
{
  "name": "Nome do Agente",
  "role": "Função na Crew",
  "goal": "Objetivo principal",
  "backstory": "Contexto e personalidade",
  "tools": ["campaign_vector_search", "campaign_stats"],
  "llm_model": "deepseek-chat",
  "temperature": 0.7,
  "max_iter": 5,
  "compliance_rules": [...]
}
```

5.3 Logging e Telemetria

Tabelas de Log:

- `agent_logs` : Pensamentos step-by-step dos agentes
- `crew_run_logs` : Execuções completas da Crew
- `radar_executions` : Status de cada fase do Radar

Visualização:

- Console em tempo real no frontend
- Polling a cada 3 segundos durante execução
- Histórico completo por execução

---

6. Estrutura de Dados

6.1 Tabelas Principais (Supabase)

Domínio Político

Tabela	Descrição	Linhas (est.)
`politicians`	Cadastro de políticos	~500
`cities`	Municípios brasileiros	~5.570
`offices`	Cargos políticos	~10
`mandates`	Vínculo político-cargo-cidade	~1.000

Domínio Campanha

Tabela	Descrição	Linhas (est.)
`campaigns`	Campanhas eleitorais	~50
`locations`	Locais de votação	~500
`location_results`	Votos por candidato	~5.000
`strategies`	Estratégias geradas	~200
`tasks`	Tarefas táticas	~1.000

Domínio Radar

Tabela	Descrição	Linhas (est.)
`promises`	Promessas extraídas	~500
`promise_verifications`	Verificações de cumprimento	~1.500
`radar_executions`	Histórico de execuções	~100
`municipal_expenses`	Dados fiscais TCESP	~50.000

Domínio IA

Tabela	Descrição	Linhas (est.)
`agents`	Configuração de agentes	~20
`personas`	Personas legadas (fallback)	~10
`agent_logs`	Logs de pensamento	~10.000
`documents`	Documentos para RAG	~100
`document_chunks`	Chunks vetorizados	~5.000

6.2 Row Level Security (RLS)

O sistema utiliza RLS do Supabase para isolamento de dados:

```
-- Exemplo: Usuários só veem campanhas vinculadas
CREATE POLICY "Users can view own campaigns" ON campaigns
FOR SELECT USING (auth.uid() = owner_id);

-- Service Role tem acesso total
CREATE POLICY "Service role full access" ON campaigns
FOR ALL TO service_role USING (true);
```

7. Mapa de Rotas e Funcionalidades

7.1 Rotas Frontend

Públicas

Rota	Status	Descrição
`/login`	✅ OK	Autenticação Supabase

Dashboard do Candidato (`/campaign/[id]`)

Rota	Status	Descrição
`/dashboard`	✅ OK	Visão central de métricas
`/map`	✅ Enterprise	Mapa Interativo Leaflet
`/plan`	✅ OK	Plano Estratégico da campanha
`/tasks`	✅ OK	Kanban de tarefas
`/promises`	✅ OK	Interface do Radar Premium

Painel Admin (`/admin`)

Rota	Status	Descrição
`/dashboard`	✅ OK	Métricas do sistema
`/candidatos`	✅ OK	CRUD de clientes
`/agentes`	✅ OK	Editor de personas IA
`/agentes/biblioteca`	✅ OK	Biblioteca de agentes
`/flows`	⚠️ Parcial	Editor Kestra (experimental)
`/radar/monitoramento`	✅ OK	Painel do Radar
`/radar/politicos`	✅ OK	Base de políticos
`/radar/cidades`	✅ OK	Cadastro de cidades

7.2 Endpoints Backend

Agentes (`/api/agents`)

Método	Endpoint	Função
GET	`/`	Lista todos os agentes
GET	`/{id}`	Detalhes de um agente
POST	`/`	Cria novo agente
PUT	`/{id}`	Atualiza agente
DELETE	`/{id}`	Remove agente

Campanhas (`/api/campaigns`)

Método	Endpoint	Função
GET	`/`	Lista campanhas
POST	`/`	Cria campanha
GET	`/{id}/metrics`	Métricas da campanha

Radar Premium (`/api/campaigns/{id}/radar/{mandate_id}`)

Método	Endpoint	Função
POST	`/phase1`	Extração de promessas
POST	`/phase2`	Cruzamento fiscal
POST	`/phase3`	Varredura de mídia
GET	`/phase-status`	Status de todas as fases

Estratégias (`/api/strategies`)

Método	Endpoint	Função
GET	`/`	Lista estratégias
PATCH	`/{id}/status`	Atualiza status
POST	`/{id}/duplicate`	Duplica estratégia

8. Benefícios por Perfil de Usuário

8.1 Para o Candidato

Benefício	Descrição	ROI Estimado
Economia de Tempo	Plano estratégico em minutos (não dias)	80% menos tempo
Decisões Baseadas em Dados	Análise real de votos por seção	+15% assertividade
Mapa de Oportunidades	Identificação de redutos eleitorais	+10% territórios
Monitoramento de Adversários	Radar de promessas dos concorrentes	Vantagem competitiva

8.2 Para o Coordenador de Campanha

Benefício	Descrição	ROI Estimado
Gestão Centralizada	Todos os dados em um lugar	-50% tempo procurando
Tarefas Automatizadas	IA gera o que fazer	-70% planejamento
Equipe Alinhada	Kanban compartilhado	+30% produtividade
Relatórios Instantâneos	Dashboards em tempo real	Tomada de decisão rápida

8.3 Para o Político em Mandato

Benefício	Descrição	ROI Estimado
Accountability	Rastreie suas promessas	Credibilidade pública
Verificação Fiscal	Prove execução orçamentária	Defesa contra fake news
Mídia Espontânea	Encontre menções positivas	Material de campanha
Planejamento de Reeleição	Dados históricos consolidados	Campanha mais forte

9. Diferenciais Competitivos

9.1 Tecnologia Proprietária

Diferencial	Descrição
Genesis Crew	Múltiplos agentes IA colaborando em tempo real
Radar de Promessas	Único sistema que cruza promessas com TCESP
Mapa Enterprise	Visualização geoespacial com camadas táticas
RAG Político	Busca semântica em documentos de campanha

9.2 Integrações Exclusivas

- ✅ **TSE** - Dados oficiais de eleições
- ✅ **TCESP** - Execução orçamentária municipal
- ✅ **Tavily** - Busca avançada na web
- ✅ **DeepSeek** - LLM de última geração

9.3 Arquitetura Moderna

- **Multi-tenant:** Isolamento total de dados por cliente
- **Serverless-ready:** Deploy em Vercel + Supabase
- **Event-driven:** Processos assíncronos não bloqueiam UI
- **Observable:** Logging completo de todas as operações

10. Roadmap e Evolução

10.1 Versão Atual (v3.0)

Status: ✅ Produção

- [x] Dashboard completo
- [x] Mapa Interativo Enterprise
- [x] Radar Premium (3 fases)

- [x] Genesis Crew operacional
- [x] Admin Panel completo
- [x] Multi-tenant RLS

10.2 Próximas Versões

v3.1 (Q1 2026)

- [ ] Modo Demo toggle no Radar
- [ ] Validação JSON de Agentes (Zod)
- [ ] Upload batch de locations (CSV)
- [ ] Testes E2E automatizados

v3.2 (Q2 2026)

- [ ] Integração WhatsApp (notificações)
- [ ] API pública para consultorias
- [ ] Dashboard comparativo multi-candidatos
- [ ] Mobile App (React Native)

v4.0 (S2 2026)

- [ ] IA Generativa para conteúdo de campanha
- [ ] Análise de sentimento de redes sociais
- [ ] Previsão de resultados (Machine Learning)
- [ ] Marketplace de agentes customizados

📊 Métricas do Projeto

Métrica	Valor
Arquivos TypeScript	~100
Arquivos Python	~50
Componentes React	~80
Endpoints API	~40
Tabelas no Banco	~25
Migrations SQL	21
Linhas de Código (est.)	~50.000

🏁 Conclusão

O **PRISMA 888** representa o estado da arte em plataformas de inteligência política no Brasil. Combinando Big Data eleitoral, Inteligência Artificial avançada e design moderno, a plataforma oferece uma vantagem competitiva significativa para candidatos, mandatários e consultorias.

Diferenciais-chave:

1. **Único** no mercado a cruzar promessas com dados fiscais reais (TCESP)
2. **Crews de IA** que colaboram para gerar estratégias completas
3. **Mapa Interativo** com inteligência territorial
4. **Arquitetura Enterprise** pronta para escala

© 2026 PRISMA 888 - Inteligência Política Enterprise Documento gerado automaticamente pelo sistema de auditoria.